

CS4031 – Compiler Construction
Course Project Proposal
Spring 2026

Project Title:	PixLang
Theme:	Pixel Art Generator

Team Members:

Student Name	Student ID
1. Mehroz Muneer	23K-0748
2. Dua Shafiq	23K-0825
3. Kashan Hamid	23K-0747

1. Language Concept

(A brief, 3-4 sentence description of your language. What is its purpose? Who is it for?)

PixLang is a domain-specific language designed to generate pixel art images through simple, readable drawing commands. It targets digital artists, game developers, and students who want a programmable, scriptable approach to creating retro-style graphics without deep programming expertise. Users can define canvases, color palettes, and drawing primitives using an English-like syntax, and the compiler produces PNG or animated GIF output files. PixLang bridges the gap between visual creativity and code, making algorithmic art generation accessible and intuitive.

2. Key Features

(List 3-5 core features your language will support.)

- Variables and arithmetic expressions with full operator precedence (+, -, *, /, MOD)
- Drawing primitives: RECT, CIRCLE, LINE, PIXEL, and FILL commands with color arguments
- Control flow: LOOP WHILE, LOOP TIMES, and IF/ELSE conditionals
- Named color palettes with indexed access (e.g., PALETTE retro: #1A1C2C, #5D275D, ...)
- Canvas configuration (CANVAS width height) and file output (SAVE "filename.png")

3. Example Program

(A small but concrete code snippet showing what the language looks like.)

```
; Concentric squares pattern
```

```
CANVAS 64 64
```

```
PALETTE retro: #1A1C2C, #5D275D, #B13E53, #EF7D57, #FFCD75, #A7F070
```

```
FILL #1A1C2C
```

```
VAR size = 60
```

```
VAR col = 1
```

```
LOOP WHILE size > 4:
```

```
  RECT (32 - size/2), (32 - size/2), size, size, retro[col]
```

```
  size = size - 8
```

```
  col = (col + 1) MOD 6
```

```
ENDLOOP
```

```
CIRCLE 32 32 RADIUS 6 COLOR retro[5]
```

```
SAVE "concentric.png"
```

4. Target Output

- PNG image file (primary output for all programs)
- Generated Python script using Pillow's ImageDraw API as the code generation backend
- Console output showing intermediate representations when --debug flag is used
- Error messages with line and column information for invalid programs
- REPL mode for testing individual drawing commands interactively

5. Expected Challenges

(Identify 2-3 potential technical or design difficulties.)

- Implementing correct operator precedence parsing for nested arithmetic expressions inside drawing command arguments (e.g., RECT (cx - size/2), ...). This requires a well-designed recursive descent parser with proper expr/term/factor non-terminals.
- Building a type system that distinguishes between INT, FLOAT, COLOR, and STRING types and catches type mismatches at compile time (e.g., passing a string where RECT expects an integer coordinate).
- Designing effective optimizations such as constant folding on loop variables and dead variable elimination while ensuring the visual output remains identical before and after optimization.